RIT | Golisano College of Computing and Information Sciences
School of Information

NSSA322 Systems Administration II

Lab 4: Web Services

Student Lab Guide — Apache origin • Nginx reverse proxy • Name-based virtual hosts • TLS (self-signed + ACME)

Group number	
Member 1 (A) name / RIT username	
Member 2 (B) name / RIT username	
Member 3 (C) name / RIT username	
Bookstack Lab 4 page URL	
Web / proxy FQDN / IP	
Origin (Apache) listen address	

Instructions

This is a group lab. Complete the work on your RLES infrastructure during the Thursday 5:00–7:00 PM session. Document as you go in your shared Bookstack. Partial credit is available for activities and documentation at the instructor's or TA's discretion.

All NSSA322 labs are scored out of 100. You must earn 70 or higher to pass this lab. Failing three labs (score below 70) results in automatic course failure, consistent with the NSSA-221 policy.

> **Do not do the following**
> Do not install the public web stack on the FreeIPA/IdM, Zabbix, Graylog, or TrueNAS hosts. Do not disable SELinux. Do not leave port 80 serving unique content after HTTPS is working — HTTP must redirect. Do not use a certificate whose CN/SAN does not match the name clients type in the browser.

Introduction

NSSA-221 taught you how to stand up Apache and a couple of name-based virtual hosts. That is the origin. In production, clients rarely talk to the origin directly. A reverse proxy terminates TLS, chooses a backend, adds request headers, and gives you one place to apply rate limits, access logs, and certificate renewal.

This lab builds that pattern on your Lab 1 DNS and directory, Lab 2 observability, and — if you finished it — Lab 3 shared storage. You will run Apache as the origin for two sites, put Nginx in front as the public HTTP/HTTPS endpoint, and protect the sites with TLS. You will attempt a Let's Encrypt (ACME) issuance if the lab network allows it, and you will always have a working internal certificate with a correct SAN so HTTPS can be demonstrated from Windows and Linux clients.

Lab Summary

Review Labs 1–3 → Provision a dedicated web VM and DNS names → Deploy Apache with two name-based virtual hosts → Deploy Nginx as a reverse proxy → Enable HTTPS with a SAN-correct certificate and HTTP-to-HTTPS redirect → Attempt ACME/Let's Encrypt or document the lab-network blocker and complete the internal-CA fallback → Hook the sites into Zabbix and Graylog → Document design decisions in Bookstack.

Goals

By the end of this lab, you will be able to:

- Deploy Apache httpd on Rocky Linux with at least two name-based virtual hosts and distinct document roots.
- Deploy Nginx as a reverse proxy that forwards Host and X-Forwarded-* headers to the origin.
- Create DNS A or CNAME records so clients reach the proxy by name, not only by IP.
- Issue a TLS certificate whose subject alternative names match the site names, install it on Nginx, and redirect HTTP to HTTPS.
- Explain the difference between a self-signed certificate, an internal CA, and a publicly trusted Let's Encrypt certificate — and show which one you actually used.
- Validate sites from a Windows browser and from curl on Linux, including certificate details.
- Add a Zabbix web scenario or HTTP check and ship access or error logs toward Graylog.
- Maintain professional collaborative documentation in your semester group Bookstack.

Prerequisites

- Labs 1 and 2 complete and still running (DNS, AD/IdM, Zabbix, Graylog).
- Lab 3 TrueNAS is recommended but not a hard blocker. If the NAS is up, you may place site content on an NFS export.
- One additional Rocky Linux 9 VM for the web stack (or two if you split origin and proxy — one VM is enough for the two-hour core).
- Windows 11 and Rocky Linux clients from earlier labs.
- Your group Bookstack already contains Labs 1–3 pages.

Pre-Lab (complete before Thursday)

- Complete the Tuesday 15-minute TLS exercise: generate a key + CSR or a self-signed cert with openssl and be able to read subject and SAN with openssl x509 -in cert.pem -noout -text.
- Decide three names before you arrive. Example: www.yourdomain, app.yourdomain, and web.yourdomain for the host itself.
- Confirm whether your group has any publicly resolvable name. Most RLES groups will not. Plan the internal-certificate path as the default.
- Skim Apache VirtualHost, Nginx proxy_pass, and the Rocky httpd/nginx SELinux booleans.

Recommended Architecture

Use one Rocky Linux 9 VM named web.yourdomain (2–4 vCPU, 4 GiB RAM). Apache listens on 127.0.0.1:8080 (origin only). Nginx listens on 0.0.0.0:80 and 0.0.0.0:443 and proxies to Apache. Clients resolve www.yourdomain and app.yourdomain to the same proxy address via A or CNAME records.

If you prefer two VMs (origin + proxy), that is an acceptable design — document the extra hop and the firewall between them. Do not put the public listener on Apache and the proxy on the same ports without a clear split; colliding :80 services will waste the lab period.

Group Roles

- Member A: VM, DNS records, Apache origin, virtual hosts, and document roots.
- Member B: Nginx reverse proxy, headers, TLS certificate, HTTPS redirect.
- Member C: ACME/Let's Encrypt attempt or internal-CA write-up, Zabbix/Graylog hook, Bookstack.

Every member must be able to explain the request path: client → DNS → Nginx → Apache → file, and where TLS is terminated.

Bookstack Requirements

- Add a Lab 4 page linked from Home. Update the architecture diagram to show DNS names, Nginx, Apache, TLS termination, and the path to Zabbix/Graylog.
- Paste redacted config excerpts (httpd vhost, nginx server block, openssl/certbot command). Never paste private keys.
- Document while you work.

Activity Summary

- Activity 0 — Infrastructure review, VM, and DNS (10–15 min)
- Activity 1 — Apache origin and name-based virtual hosts (25–30 min)
- Activity 2 — Nginx reverse proxy (20–25 min)
- Activity 3 — TLS, redirect, and ACME/Let's Encrypt attempt (25–30 min)
- Activity 4 — Observability hook and operational reflection (15–20 min)

Two-hour target. Core checkpoint if time runs short: two vhosts distinguishable by name, Nginx proxying both, HTTPS with a SAN-correct cert, HTTP redirect, one remote browser screenshot, Bookstack started.

Tuesday lecture tie-in (15 minutes)

In pairs, generate a key and a certificate (or CSR) with openssl. Include two DNS names in the subject alternative name extension. Inspect the cert with openssl x509 and explain why a CN-only certificate is no longer enough for modern browsers. You will install a SAN-correct certificate on Nginx in this lab.

Activity 0: Infrastructure Review, VM, and DNS

Purpose

Name-based virtual hosts and TLS both fail if DNS is wrong. Confirm Labs 1–2, then give the web VM a stable address and names.

Procedure

1. Verify DNS, the pfSense gateway, and that Zabbix and Graylog still load.
2. Provision web.yourdomain. Assign a static address or reservation. Update the VM and confirm outbound HTTP/HTTPS for package installs.

3. Create DNS records: an A record for web.yourdomain, plus A or CNAME records for www.yourdomain and app.yourdomain pointing at the same address. Add PTRs if your zone design from Lab 1 uses them.
4. From a Windows client and a Rocky client, resolve all three names before you install packages.

Checkpoint

- dig or Resolve-DnsName returns the expected address for www and app.
- The web VM can reach your repos (dnf makecache succeeds).

Required evidence

- Figure 1 — DNS Validation. Output of dig www.yourdomain + dig app.yourdomain (or PowerShell equivalent) with hostname and date.

Activity 1: Apache Origin and Virtual Hosts

Purpose

Apache is the origin. It serves two different sites based on Host. It should not be the public TLS terminator in the core design.

Procedure

1. Install httpd and mod_ssl (you will use Apache only on HTTP for the core path). Enable and start httpd. Keep SELinux Enforcing.
2. Rebind the origin. Set Apache to listen on 127.0.0.1:8080 (Listen 127.0.0.1:8080) so Nginx owns :80 and :443. Confirm with ss -lntp | grep httpd.
3. Create two document roots, for example /var/www/www.yourdomain and /var/www/app.yourdomain, each with a distinct index.html that names the site and the group. If Lab 3 NFS is available, one root may live on the NAS — document that choice.
4. Add two VirtualHost *:8080 blocks (files under /etc/httpd/conf.d/). Each needs ServerName, DocumentRoot, and its own access/error log. Add a <Directory> block that allows the custom path.
5. Fix SELinux labels if you left the default tree (restorecon -Rv /var/www) or set httpd_sys_content_t on a custom path. Open nothing on :8080 in firewalld — origin stays local.
6. Run apachectl configtest and httpd -S. Prove locally: curl -H 'Host: www.yourdomain' http://127.0.0.1:8080 and the same for app. The bodies must differ.

Checkpoint

- httpd -S lists both name-based vhosts on :8080.
- Local curls return different pages for the two Host headers.

Required evidence

- Figure 2 — Apache Virtual Hosts. Combined output of hostname; date; apachectl configtest; httpd -S.
- Figure 3 — Origin Content Check. Both curl -H Host commands and their page bodies.

Activity 2: Nginx Reverse Proxy

Purpose

Nginx is what clients hit. It chooses the backend from the request Host header, forwards the original host and client address, and will later terminate TLS.

Procedure

1. Install nginx. Enable and start it. Confirm it is listening on :80.
2. Create two server blocks (files under /etc/nginx/conf.d/), one for www.yourdomain and one for app.yourdomain. Each should listen 80 and proxy_pass http://127.0.0.1:8080;.
3. Set proxy headers on both blocks: Host $host, X-Real-IP $remote_addr, X-Forwarded-For $proxy_add_x_forwarded_for, and X-Forwarded-Proto $scheme. Without these, the origin logs the proxy as the client and cannot build correct redirects later.
4. Allow HTTP in firewalld (and HTTPS now, so Activity 3 is not blocked): firewall-cmd --permanent --add-service=http --add-service=https && firewall-cmd --reload.
5. Allow Nginx to talk to Apache under SELinux: setsebool -P httpd_can_network_connect 1 (covers the proxy origin call on many Rocky builds). If proxy_pass fails with 502, check audit.log / ausearch and nginx error.log before disabling SELinux.
6. Run nginx -t and reload. From a second VM, curl -I http://www.yourdomain and curl http://app.yourdomain. Confirm you get the matching origin page, not the default Nginx page.

Checkpoint

- Remote curls by name return the correct site body for both names.
- Apache access logs show the forwarded client address or at least the expected Host.

Required evidence

- Figure 4 — Proxy Validation. Remote curl of both HTTP names, plus a snippet of the nginx server block (no secrets).

Activity 3: TLS, Redirect, and ACME

Purpose

A site on port 80 is unfinished. You will terminate TLS on Nginx with a certificate whose SAN matches both site names (or one cert per name), redirect HTTP to HTTPS, and attempt a publicly trusted issuance.

Procedure — required internal certificate

1. Create a certificate that a browser can match by name. Minimum: one cert with SAN DNS:www.yourdomain and DNS:app.yourdomain, or two single-name certs. Use openssl as in Tuesday lecture. A CN-only cert without SAN will fail in current browsers — do not ship that.
2. Install the cert and key under /etc/pki/nginx/ (or /etc/nginx/ssl/). Permissions: key 600, owned by root. Point each Nginx server block at listen 443 ssl; with ssl_certificate and ssl_certificate_key. Enable a modern TLS policy (TLS 1.2+).
3. On the port-80 server blocks, replace the proxy with a permanent redirect to HTTPS (return 301 https://$host$request_uri;). HTTPS blocks keep the proxy_pass to Apache.
4. nginx -t && systemctl reload nginx. From a client: curl -kI https://www.yourdomain and curl -kI http://www.yourdomain. HTTP must be 301/302 to HTTPS. Then openssl s_client -connect web.yourdomain:443 -servername www.yourdomain and confirm the SAN list.

5. From Windows 11, open https://www.yourdomain and https://app.yourdomain. You will see a trust warning for a self-signed cert. That is expected unless you imported an internal CA. Capture the warning and the page — both are evidence.

Procedure — Let's Encrypt / ACME attempt

6. Install certbot from EPEL (dnf install epel-release && dnf install certbot python3-certbot-nginx). Do not fight snap unless the instructor posted that method.
7. If www.yourdomain is publicly resolvable and port 80 on the proxy is reachable from the Internet, run certbot --nginx -d www.yourdomain -d app.yourdomain and complete issuance. Record the live certificate issuer (Let's Encrypt).
8. If public HTTP-01 is impossible — the normal RLES case — run a dry-run or a real attempt once, capture the exact error, and complete the internal path above. Then write six to eight sentences in Bookstack: what ACME HTTP-01 needs, why this lab network cannot satisfy it, and how Let's Encrypt would change operations (auto-renew, public trust, no browser warning).
9. Optional stronger path (extension): stand up an internal ACME CA (step-ca) and issue the Nginx cert through ACME anyway. That still proves you understand the protocol. It is not required to pass.

Checkpoint

- openssl s_client (or the browser certificate view) shows SAN names that match the URLs you type.
- HTTP requests redirect to HTTPS.
- Both sites load over HTTPS from a remote client.
- You either have a Let's Encrypt cert, or a captured ACME failure plus the written explanation.

Required evidence

- Figure 5 — Certificate SAN. openssl x509 or s_client output showing subject and SAN.
- Figure 6 — HTTPS and Redirect. curl -kI of http and https for at least one name.
- Figure 7 — Remote Browser HTTPS. Windows (or other remote) browser on both sites. Include the address bar.
- Figure 8 — ACME Result. Successful certbot output, or the failure message plus the Bookstack paragraph.

Core required tonight vs. extension homework

Core: two Apache vhosts on :8080, Nginx proxy on :80/:443, SAN-correct TLS, HTTP redirect, remote proof of both sites, ACME attempt or documented blocker, Zabbix or Graylog hook, Bookstack.

Extension: split origin and proxy onto two VMs; internal ACME CA; HTTP public key pinning discussion; TLS 1.3-only policy; site content served from the Lab 3 NFS dataset.

Activity 4: Observability Hook and Reflection

Purpose

A web stack nobody watches will fail quietly. Tie the new sites back to Lab 2.

Procedure

1. In Zabbix, add a web scenario or HTTP agent item against https://www.yourdomain (ignore TLS errors if you are still on a self-signed cert, and document that setting). Confirm a successful check.

2. Ship Nginx or Apache access/error logs to Graylog. The Lab 2 rsyslog forwarder on this VM is enough if you include /var/log/nginx/access.log. Generate a request and find it in Graylog.
3. As a group, walk one failure: certificate expired, origin down (502), or wrong Host header. Who notices first — the user, Zabbix, or Graylog?

Checkpoint

- Zabbix shows a recent successful web check, or Graylog shows a proxied request you generated.

Required evidence

- Figure 9 — Observability Hook. Zabbix web scenario/item or Graylog search of a web log line.

Required Figures (submit on Bookstack)

Use these exact titles. Caption every figure. Include hostname and date on terminal captures.

- Figure 1 — DNS Validation
- Figure 2 — Apache Virtual Hosts
- Figure 3 — Origin Content Check
- Figure 4 — Proxy Validation
- Figure 5 — Certificate SAN
- Figure 6 — HTTPS and Redirect
- Figure 7 — Remote Browser HTTPS
- Figure 8 — ACME Result
- Figure 9 — Observability Hook

Reflection Questions

Answer on the Bookstack Lab 4 page. A few precise sentences each, with evidence.

- Q1. Why did Apache listen on 127.0.0.1:8080 instead of :80? What does that buy you operationally?
- Q2. Which four proxy headers did you set, and what breaks if Host or X-Forwarded-Proto is missing?
- Q3. Why is a CN-only certificate insufficient? Quote the SAN list from Figure 5.
- Q4. What did your Let’s Encrypt / ACME attempt prove? If it failed, what condition would have to change in DNS or the firewall for HTTP-01 to succeed?
- Q5. Where is TLS terminated in your design, and how would you change that if Apache needed the client certificate later?

What You Submit

- Bookstack Lab 4 page URL.
- Updated architecture diagram (DNS, Nginx, Apache, TLS termination).
- Figures 1–9 with captions.
- Answers to Q1–Q5.
- Redacted httpd and nginx config excerpts. No private keys.

- Troubleshooting journal: at least two detailed entries.
- Individual contribution notes.

Bookstack is the official deliverable. Do not upload private keys to Bookstack or myCourses.

Conclusion

You moved from “Apache serves a page” to an origin-plus-proxy design with name-based routing and TLS. That is the pattern used in front of application servers, containers, and the mail or wiki services you may run later. Leave the sites up. Lab 5 will expect DNS names that already work, and Lab 6 may put an application behind this same proxy.

Rubric

Total 100 points. Passing threshold is 70.

Item	Points	Sign-off
DNS names resolve to the web/proxy host (Fig. 1)	**10**	
Apache: two distinct name-based vhosts on the origin port (Fig. 2–3)	**15**	
Nginx reverse proxy serves both sites by name over HTTP (Fig. 4)	**15**	
TLS installed with matching SAN; HTTP redirects to HTTPS (Fig. 5–6)	**20**	
Remote browser proof of both HTTPS sites (Fig. 7)	**10**	
ACME/Let’s Encrypt success or documented HTTP-01 blocker (Fig. 8)	**10**	
Bookstack: diagram, Q1–Q5, journal, contributions, Fig. 9 hook	**20**	
Total	**100**	

Common Pitfalls & Troubleshooting

- Both names return the same default page — ServerName does not match what the client sent, or Nginx is not sending Host $host.
- 502 Bad Gateway — Apache is not listening on 127.0.0.1:8080, or SELinux denied the proxy connect. Check ss, nginx error.log, and ausearch -m avc -ts recent.
- Browser says NET::ERR_CERT_COMMON_NAME_INVALID — the cert SAN does not include the exact name in the address bar.
- certbot fails immediately — expected on a private lab network. Capture the error; do not burn the period retrying HTTP-01 against 192.168.0.0/16.
- HTTP still serves content after you “enabled TLS” — you added listen 443 but never redirected :80.
- Permission denied on DocumentRoot — SELinux label, not chmod. Use restorecon or semanage fcontext.
- Firewall allows 80 but not 443 — Activity 3 looks like a dead host from Windows.
- You published the private key in Bookstack — treat that as a failed control. Rotate the key and cert.

Optional Extension (homework)

- Split Nginx and Apache onto two VMs and restrict Apache to the proxy’s address only.

- Issue the server certificate from an internal ACME CA (step-ca) and import the CA on one Windows client so the warning disappears.
- Put one document root on the Lab 3 NFS export and prove the site survives a web VM reboot via _netdev.
- Add a Zabbix trigger on TLS certificate expiry days.

Primary Resources

Apache HTTP Server 2.4 virtual hosts: https://httpd.apache.org/docs/2.4/vhosts/

Nginx reverse proxy: https://nginx.org/en/docs/http/ngx_http_proxy_module.html

Rocky Linux — SSL keys and Let's Encrypt / certbot: https://docs.rockylinux.org/guides/security/generating_ssl_keys_lets_encrypt/

Let's Encrypt — how HTTP-01 works: https://letsencrypt.org/docs/challenge-types/

openssl-req / x509 man pages on the web VM (man openssl-req, man x509v3_config).

Excellent work — clients now reach named sites over TLS through a proxy, not a bare origin.